

Maulana Azad National Institute of Technology (MANIT)



Department of Computer Science and Engineering

Session: 2025–26

“MINOR-PROJECT REPORT”

Deepfake Detection Using Graph Convolutional Networks (GCN) and Convolutional Neural Networks (CNN)

*Submitted in partial fulfillment for the degree of Bachelor of Technology in Computer
Science and Engineering*

SUBMITTED BY:

Abhijeet Singh Gour (2311201368)
Lokesh Vaishnav (2311201371)
Neeraj Choitwani (2311201337)
Adarsh Patel (2311201344)

GUIDED BY:

Dr. JAYTRILOK CHOUDHARY
(Dept. of Computer Science
& Engineering, MANIT Bhopal)

Maulana Azad National Institute of Technology – Bhopal

Department of Computer Science and Engineering

DECLARATION

I hereby declare that the work presented in this Project Report, entitled “**Deepfake Detection Using Graph Convolutional Networks (GCN) and Convolutional Neural Networks (CNN)**”, is submitted in partial fulfilment of the requirements for the award of the degree in the Department of Computer Science and Engineering, **Maulana Azad National Institute of Technology, Bhopal**.

It is an authentic record of my work carried out from **January 2026 to April 2026**, under the guidance of my supervisor. This project and its report, in part or whole, have not been presented or submitted for any purpose in any other institute or organization.

Abhijeet Singh Gour (2311201368)

Lokesh Vaishnav (2311201371)

Neeraj Choitwani (2311201337)

Adarsh Patel (2311201344)

Place: MANIT Bhopal

Date: April 2026

Maulana Azad National Institute of Technology – Bhopal

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that **Abhijeet Singh Gour, Lokesh Vaishnav, Neeraj Choitwani,** and **Adarsh Patel**, students of B.Tech 3rd Year (Computer Science & Engineering), have successfully completed their project titled “**Deepfake Detection Using Graph Convolutional Networks (GCN) and Convolutional Neural Networks (CNN)**” in partial fulfilment of their Bachelor of Technology in Computer Science & Engineering.

Supervisor Name:

Dr. JAYTRILOK CHOUDHARY

Department of Computer Science & Engineering

MANIT Bhopal

ACKNOWLEDGEMENT

With due respect, we express our deep sense of gratitude to our respected Guide and Project Coordinator, **Dr. JAYTRILOK CHOUDHARY**, for their valuable help and guidance throughout the course of this project. We are sincerely thankful for their encouragement, rigorous evaluation, and constructive criticism, which were instrumental in the successful completion of this work.

We are also grateful to our respected Director, **Dr. Karunesh Kumar Shukla**, for permitting us to utilize all the necessary college facilities. We also extend our gratitude to our HOD, **Dr. R.K Pateriya**, for allowing us to use the departmental laboratories and other services.

We are thankful to all faculty members, staff, and laboratory attendants of our department for their cooperation and support. Last but not least, we express deep appreciation to our family members and batchmates for their constant encouragement.

Place: MANIT Bhopal

Abhijeet Singh Gour (2311201368)

Lokesh Vaishnav (2311201371)

Neeraj Choitwani (2311201337)

Date: April 2026

Adarsh Patel (2311201344)

ABSTRACT

In recent years, the proliferation of deepfake content—digitally altered media that convincingly manipulates human appearances and voices—has posed significant challenges across social, ethical, and cybersecurity domains. Traditional deepfake detection methods, which primarily rely on Convolutional Neural Networks (CNNs), often struggle to capture subtle facial irregularities and spatial relationships in fine detail.

To address this limitation, this project proposes a **hybrid model** that combines Graph Convolutional Networks (GCNs) with CNNs. The GCN branch processes facial landmarks extracted using *dlib*'s 68-point detector, organized into graphs via Delaunay triangulation, thereby capturing relational and spatial information between landmarks. Simultaneously, the CNN branch extracts pixel-level textural features from the input image. The outputs of both branches are concatenated and passed through fully connected layers for final classification.

The model is evaluated on the publicly available **140K Real and Fake Faces** dataset from Kaggle, using a 20,000-image subset (10,000 real and 10,000 fake). The proposed hybrid model achieves a **test accuracy of 93.89%**, outperforming standalone CNN (91.8%) and GCN-only (87.4%) approaches, and is competitive with state-of-the-art architectures such as InceptionV3 (93.7%) and VGG16 (92.4%). Cross-dataset validation on the 50K Celebrity Faces dataset further confirms strong generalization at 92% accuracy.

The model also demonstrates robustness under JPEG compression, Gaussian noise, and occlusion perturbations. This work contributes a novel and effective hybrid framework that exploits both geometric and textural cues for reliable deepfake detection.

Keywords: Deepfake Detection, Graph Convolutional Networks (GCN), Convolutional Neural Networks (CNN), Facial Landmarks, Delaunay Triangulation, Hybrid Model, Spatial Relationships, Pixel-Level Analysis.

Contents

Abbreviations / Nomenclature	v
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objective	2
1.4 Scope of the Project	2
2 Literature Review	3
2.1 Overview of Deepfake Technology	3
2.2 CNN-Based Deepfake Detection	3
2.3 Graph Neural Networks in Face Analysis	4
2.4 Hybrid CNN + GCN Approaches	4
2.5 Identified Research Gap	4
3 System Architecture and Design	6
3.1 Architectural Overview	6
3.2 Dataset Description	7
3.3 Preprocessing Pipeline	8
3.3.1 Image Preprocessing (CNN Branch)	8
3.3.2 Facial Landmark Extraction and Graph Construction (GCN Branch)	8
4 Methodology	10
4.1 GCN Branch Architecture	10
4.2 CNN Branch Architecture	10
4.3 Fusion and Classification Layer	11
4.4 Training Configuration	12
4.5 Evaluation Metrics	12
5 Implementation and Code	13
5.1 Environment Setup	13
5.2 Data Preprocessing	13
5.3 GCN Layer Implementation	14

5.4	Hybrid GCN + CNN Model	15
5.5	Training the Model	16
5.6	Robustness Evaluation	17
6	Results and Discussion	19
6.1	Accuracy Metrics	19
6.2	Ablation Study	19
6.3	Comparison with Baseline Models	20
6.4	Confusion Matrix Analysis	20
6.5	Classification Report	20
6.6	Robustness Evaluation	21
6.7	Cross-Dataset Validation	21
6.8	Training Curves	22
7	Conclusion and Future Scope	23
7.1	Conclusion	23
7.2	Limitations	23
7.3	Future Scope	24
A	Appendix: Full Model Summary and Setup	27
A.1	Complete Model Summary Output	27
A.2	Reproducibility Details	28

List of Figures

3.1	Overall architecture of the proposed Hybrid GCN+CNN deepfake detection system.	7
3.2	(Left) 68 facial landmark points detected using dlib. (Right) Delaunay triangulation-based graph construction connecting spatially adjacent landmarks.	9
6.1	Training and Validation Accuracy over 10 training epochs. The close correspondence between curves indicates no significant overfitting.	22
6.2	Training and Validation Loss over 10 training epochs. Steady decrease confirms successful convergence.	22

List of Tables

3.1	Dataset Split	7
4.1	CNN Branch Layer Summary	11
4.2	Training Hyperparameters	12
6.1	Final Accuracy Metrics	19
6.2	Ablation Study with Extended Evaluation Metrics	19
6.3	Comparison of Test Accuracy on the 140K Real and Fake Faces Dataset .	20
6.4	Confusion Matrix on Test Set (20,000 images)	20
6.5	Classification Report	20
6.6	Robustness Under Common Image Corruptions	21
6.7	Cross-Dataset Validation Results	21
A.1	Software Environment and Random Seeds	28

Abbreviations / Nomenclature

Abbreviation	Full Form
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
GCN	Graph Convolutional Network
GAN	Generative Adversarial Network
ReLU	Rectified Linear Unit
Adam	Adaptive Moment Estimation (Optimizer)
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
PR	Precision-Recall
ECE	Expected Calibration Error
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
F1	Harmonic Mean of Precision and Recall
GUI	Graphical User Interface
HPC	High-Performance Computing
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation

Chapter 1

Introduction

1.1 Background

Deepfake technology has rapidly advanced over the past decade, producing synthetic images and videos that closely mimic real human appearances and expressions. Powered by Generative Adversarial Networks (GANs) and other deep generative models, deepfakes have found legitimate applications in entertainment, visual effects, and education. However, they also pose serious risks: from spreading political misinformation to enabling identity theft, non-consensual intimate imagery, and fraud.

The challenge of detecting deepfakes has attracted significant research attention. Most early detection systems relied on Convolutional Neural Networks (CNNs) for pixel-level artifact analysis. While CNNs are excellent at recognizing local textures and patterns, they tend to overlook the *structural and relational* properties of a face — such as the geometric consistency of facial landmarks and their spatial relationships. This makes CNN-only approaches vulnerable to high-quality deepfakes that preserve pixel-level realism but subtly distort facial geometry.

Graph Convolutional Networks (GCNs), on the other hand, are specifically designed to operate on non-Euclidean, graph-structured data. Representing facial landmarks as nodes in a graph and modeling their geometric relationships as edges provides a complementary and powerful signal for deepfake detection.

1.2 Problem Statement

Traditional deepfake detection methods that rely solely on CNNs are limited in their ability to capture subtle facial irregularities and the spatial relationships between facial features. As deepfake generation techniques become increasingly sophisticated, pixel-level cues alone are insufficient for reliable detection. There is a pressing need for a detection framework that combines low-level textural analysis with high-level structural reasoning over facial geometry.

1.3 Objective

The primary objectives of this project are:

- To design and implement a **hybrid deepfake detection model** combining a Graph Convolutional Network (GCN) and a Convolutional Neural Network (CNN).
- To extract 68 facial landmarks using *dlib* and model their spatial relationships via Delaunay triangulation for GCN processing.
- To combine pixel-level image features (CNN) with structural landmark-based features (GCN) through a fusion layer for robust classification.
- To evaluate the model on the 140K Real and Fake Faces Kaggle dataset and benchmark it against standard CNN architectures.
- To assess model robustness under common image degradations including JPEG compression, Gaussian noise, and occlusion.
- To perform cross-dataset validation to evaluate generalizability.

1.4 Scope of the Project

- Development of a two-branch deep learning architecture (GCN + CNN) for binary classification of images as real or fake.
- Facial landmark extraction and graph construction using Delaunay triangulation as input to the GCN branch.
- Image preprocessing (resizing, normalization) for the CNN branch.
- Training and evaluation on a balanced 20,000-image subset of the Kaggle 140K dataset.
- Ablation study comparing CNN-only, GCN-only, and hybrid configurations.
- Robustness testing under JPEG compression, additive Gaussian noise, and random occlusion.
- Cross-dataset validation on the 50K Celebrity Faces dataset.
- End-to-end reproducible pipeline with open-source tools (Python, TensorFlow/Keras, dlib, NetworkX).

Chapter 2

Literature Review

2.1 Overview of Deepfake Technology

Deepfakes are synthetic media generated by deep learning models, most commonly Generative Adversarial Networks (GANs). A GAN consists of a *generator* that produces synthetic images and a *discriminator* that attempts to distinguish them from real ones. Through adversarial training, the generator progressively improves until its output is indistinguishable to the discriminator.

Modern deepfake generation methods — including FaceSwap, DeepFaceLab, and diffusion-based approaches — can produce highly photorealistic faces that are difficult for humans to identify. This motivates the development of automated and robust detection systems.

2.2 CNN-Based Deepfake Detection

The earliest automated detection systems employed CNN architectures to identify pixel-level artifacts introduced during deepfake generation. These include:

- **VGG16**: A deep 16-layer CNN pre-trained on ImageNet, fine-tuned for deepfake detection. Achieves 92.4% on the 140K dataset [1].
- **InceptionV3**: A wider, factorized CNN achieving 93.7% on the same dataset [1].
- **MobileNetV2**: A lightweight, efficient architecture achieving 91.5% [1].
- **XceptionNet**: A depthwise separable CNN variant frequently used as a deepfake detection baseline.

While these models perform well on in-distribution data, they are susceptible to subtle generation artifacts being masked by post-processing, and they do not explicitly model the geometry of facial structure.

2.3 Graph Neural Networks in Face Analysis

Graph Convolutional Networks (GCNs) have demonstrated promise in tasks involving structured, non-Euclidean data. In the context of face analysis, facial landmarks naturally form a graph where nodes represent anatomical keypoints and edges encode their spatial relationships.

Hsu et al. [4] proposed GRACE, a graph-regularized attentive CNN that uses Laplacian smoothing for robust video deepfake detection. Khormali and Yuan [5] developed a self-supervised graph transformer for deepfake detection, learning generalizable structural representations without relying on labeled data. She et al. [6] showed that GNNs can improve the generalization capability of deepfake detectors across unseen datasets. El-Gayar et al. [7] proposed a graph neural network approach for video deepfake detection at the frame level.

2.4 Hybrid CNN + GCN Approaches

The motivation for combining CNNs with GCNs is to harness complementary information: CNNs capture local texture and color artifacts, while GCNs capture global geometric inconsistencies in facial structure. Such hybrid approaches have been explored in face recognition, action recognition, and medical imaging. Applying this hybrid paradigm to deepfake detection represents a natural extension that has not yet been fully exploited.

2.5 Identified Research Gap

Despite rapid advances, several challenges remain:

- **Overreliance on pixel-level cues:** CNN-only models miss structural facial inconsistencies that high-quality deepfakes may preserve at the pixel level.
- **Limited geometric reasoning:** Few models explicitly encode the spatial relationships between facial landmarks.
- **Robustness to corruptions:** Many models degrade significantly under compression, noise, or occlusion.
- **Cross-dataset generalization:** Models trained on one dataset often perform poorly on deepfakes generated by different methods.
- **Temporal information:** Frame-by-frame approaches discard temporal inconsistencies present in deepfake videos.

- **Landmark detection failure:** Under extreme poses, occlusion, or poor lighting, landmark detectors may fail, degrading the GCN branch.

The proposed hybrid GCN+CNN model addresses the first two gaps directly, while also providing an evaluation framework for robustness and cross-dataset generalization.

Chapter 3

System Architecture and Design

3.1 Architectural Overview

The proposed system consists of four interconnected modules:

1. **Data Preprocessing:** Raw images are resized to 64×64 pixels and normalized. Facial landmarks are extracted using *dlib*'s 68-point predictor, and graphs are constructed via Delaunay triangulation.
2. **GCN Branch:** Processes the facial landmark graph to extract structural and relational features.
3. **CNN Branch:** Processes the raw image to extract pixel-level textural features through convolutional operations.
4. **Fusion and Classification:** Outputs from both branches are concatenated and passed through dense layers with a softmax activation for final binary classification (Real / Fake).

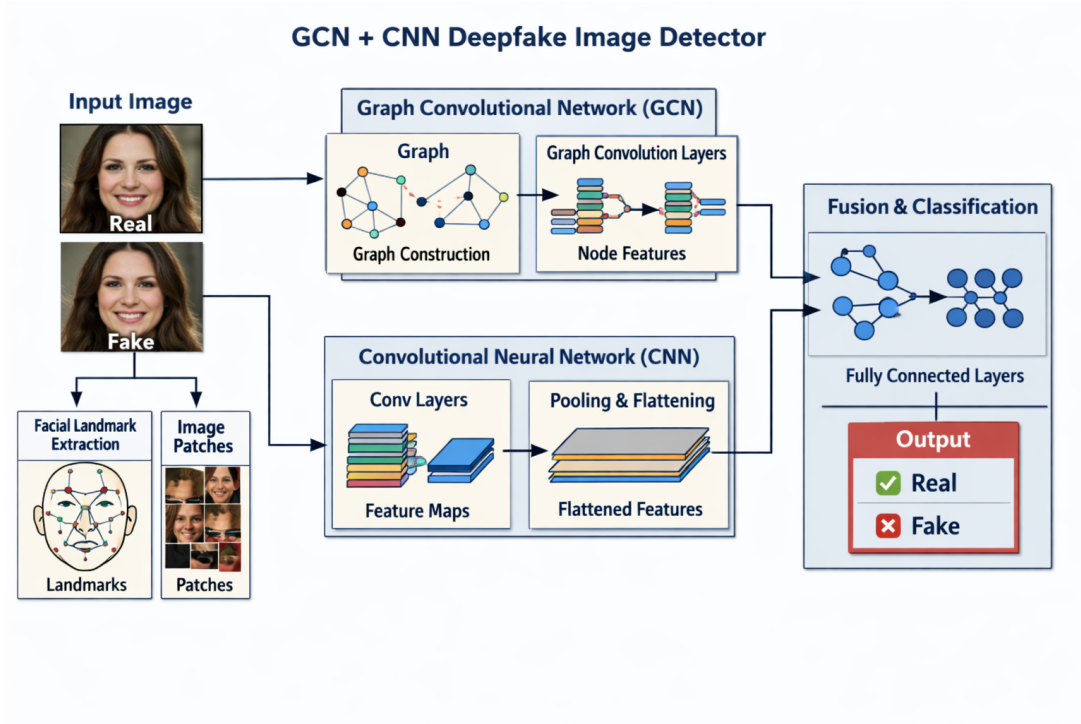


Figure 3.1: Overall architecture of the proposed Hybrid GCN+CNN deepfake detection system.

3.2 Dataset Description

The dataset used in this project is the publicly available **140K Real and Fake Faces** dataset [2], hosted on Kaggle. It consists of:

- **Total images:** 140,000 (70,000 real + 70,000 fake)
- **Real images:** Sourced from Flickr
- **Fake images:** Generated using Generative Adversarial Networks (GANs)
- **Class balance:** Equal numbers of real and fake images, suitable for binary classification

For computational efficiency while maintaining class balance, a **subset of 20,000 images** (10,000 real + 10,000 fake) was used. The data was divided as follows:

Table 3.1: Dataset Split

Split	Percentage	Number of Images
Training	70%	14,000
Validation	15%	3,000
Testing	15%	3,000

Identity separation was enforced: no individual appearing in the training set was included in the test set, ensuring a rigorous evaluation.

3.3 Preprocessing Pipeline

3.3.1 Image Preprocessing (CNN Branch)

- Images are resized to **64×64 pixels**.
- Pixel values are normalized to the range $[0, 1]$.
- Color channels are retained (RGB, 3 channels).

3.3.2 Facial Landmark Extraction and Graph Construction (GCN Branch)

1. **Landmark extraction:** *dlib*'s 68-point facial landmark detector identifies key anatomical facial points (eyes, nose, mouth, jaw, eyebrows).
2. **Graph construction:** The 68 landmarks are treated as **nodes** in an undirected graph. Edges are formed using **Delaunay triangulation**, which connects spatially neighboring landmarks.
3. **Adjacency matrix:** An $N \times N$ adjacency matrix A encodes the graph topology.
4. **Node features:** The normalized 2D coordinates (x, y) of each landmark serve as node feature vectors.

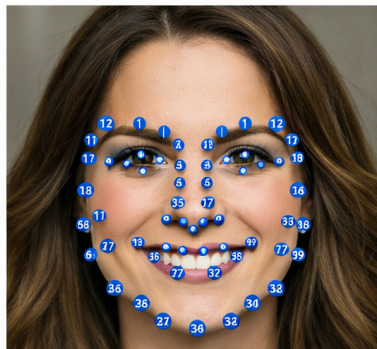
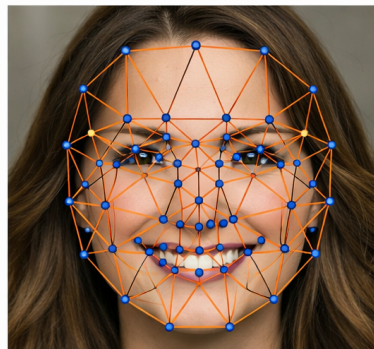
**Facial Landmarks (68 Points)****Delaunay Triangulation Graph**

Figure: Facial Landmarks (68 Points) and Delaunay Triangulation Graph

Figure 3.2: (Left) 68 facial landmark points detected using dlib. (Right) Delaunay triangulation-based graph construction connecting spatially adjacent landmarks.

Chapter 4

Methodology

4.1 GCN Branch Architecture

The GCN branch processes the facial landmark graph. Each graph convolutional layer aggregates features from neighboring nodes according to the following update rule:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right) \quad (4.1)$$

where:

- $H^{(l)}$ is the node feature matrix at layer l .
- $\tilde{A} = A + I$ is the adjacency matrix with self-loops added.
- $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$.
- $W^{(l)}$ is the trainable weight matrix for layer l .
- σ is the ReLU activation function.

The GCN branch consists of:

- **GCN Layer 1:** 64 units, aggregates information from 1-hop neighbors.
- **GCN Layer 2:** 32 units, refines representations with 2-hop neighborhood context.
- **Global Mean Pooling:** Reduces variable-length node outputs to a fixed-size feature vector.

4.2 CNN Branch Architecture

The CNN branch processes the 64×64 RGB input image. The feature extraction formula for each convolutional layer is:

$$X_{\text{out}} = \sigma(W * X_{\text{in}} + b) \quad (4.2)$$

where W and b are learned filters and biases, $*$ denotes convolution, and σ is the ReLU activation.

The CNN branch consists of:

- **Conv Block 1:** 16 filters (3×3), ReLU, MaxPooling (2×2)
- **Conv Block 2:** 32 filters (3×3), ReLU, MaxPooling (2×2)
- **Conv Block 3:** 64 filters (3×3), ReLU, MaxPooling (2×2)
- **Global Average Pooling:** Converts the final feature map to a fixed-size vector.

Table 4.1: CNN Branch Layer Summary

Layer	Output Shape	Parameters
Input ($64 \times 64 \times 3$)	(64, 64, 3)	0
Conv2D (16 filters, 3×3)	(62, 62, 16)	448
MaxPooling2D (2×2)	(31, 31, 16)	0
Conv2D (32 filters, 3×3)	(29, 29, 32)	4,640
MaxPooling2D (2×2)	(14, 14, 32)	0
Conv2D (64 filters, 3×3)	(12, 12, 64)	18,496
MaxPooling2D (2×2)	(6, 6, 64)	0
GlobalAveragePooling2D	(64,)	0

4.3 Fusion and Classification Layer

The outputs of the GCN and CNN branches are concatenated to form a unified feature representation:

$$X_{\text{fused}} = \text{concat}(X_{\text{GCN}}, X_{\text{CNN}}) \quad (4.3)$$

This fused vector is passed through:

- **Dense Layer:** 128 units, ReLU activation
- **Output Layer:** 2 units, Softmax activation

The final prediction is:

$$\hat{y} = \text{softmax}(W X_{\text{fused}} + b) \quad (4.4)$$

4.4 Training Configuration

Table 4.2: Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.001
Loss Function	Sparse Categorical Cross-Entropy
Batch Size	32
Epochs	10
Dropout Rate	0.3

4.5 Evaluation Metrics

The model is evaluated using:

- **Accuracy:** Overall fraction of correctly classified samples.
- **Precision, Recall, F1-Score:** Per-class and macro-averaged metrics.
- **ROC-AUC:** Area under the Receiver Operating Characteristic curve.
- **PR-AUC:** Area under the Precision-Recall curve.
- **ECE (Expected Calibration Error):** Measures calibration quality of predicted probabilities.
- **Confusion Matrix:** Breakdown of TP, TN, FP, FN.

Confidence intervals (95%) for ROC-AUC and PR-AUC were computed via bootstrapping on the test set.

Chapter 5

Implementation and Code

5.1 Environment Setup

```
1 pip install tensorflow==2.13.1 keras opencv-python dlib
2 pip install networkx scipy scikit-learn matplotlib seaborn
```

Listing 5.1: Installing required libraries

5.2 Data Preprocessing

```
1 import cv2
2 import dlib
3 import numpy as np
4 from scipy.spatial import Delaunay
5
6 # Load dlib's face detector and landmark predictor
7 detector = dlib.get_frontal_face_detector()
8 predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.
   dat")
9
10 def extract_landmarks(image_path):
11     """Extract 68 facial landmarks from an image."""
12     img = cv2.imread(image_path)
13     img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
14     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
15
16     faces = detector(gray)
17     if len(faces) == 0:
18         return None, None # No face detected
19
20     shape = predictor(gray, faces[0])
21     landmarks = np.array([[p.x, p.y] for p in shape.parts()])
22
23     # Normalize coordinates to [0, 1]
24     landmarks = landmarks / np.array([img.shape[1], img.shape[0]])
25     return landmarks, img_rgb
26
```

```

27 def build_graph(landmarks):
28     """Build adjacency matrix via Delaunay triangulation."""
29     tri = Delaunay(landmarks)
30     n = len(landmarks)
31     adj = np.zeros((n, n), dtype=np.float32)
32
33     for simplex in tri.simplices:
34         for i in range(3):
35             for j in range(3):
36                 if i != j:
37                     adj[simplex[i], simplex[j]] = 1.0
38
39     # Add self-loops (A_tilde = A + I)
40     adj += np.eye(n, dtype=np.float32)
41
42     # Symmetric normalization:  $D^{-1/2} A D^{-1/2}$ 
43     deg = adj.sum(axis=1)
44     d_inv_sqrt = np.diag(1.0 / np.sqrt(deg))
45     adj_norm = d_inv_sqrt @ adj @ d_inv_sqrt
46
47     return adj_norm
48
49 def preprocess_image(image_path, size=(64, 64)):
50     """Resize and normalize image for CNN input."""
51     img = cv2.imread(image_path)
52     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
53     img = cv2.resize(img, size)
54     img = img.astype(np.float32) / 255.0
55     return img

```

Listing 5.2: Image preprocessing and facial landmark extraction

5.3 GCN Layer Implementation

```

1 import tensorflow as tf
2 from tensorflow.keras.layers import Layer
3
4 class GCNLayer(Layer):
5     """Graph Convolutional Network Layer."""
6
7     def __init__(self, units, activation='relu', **kwargs):
8         super(GCNLayer, self).__init__(**kwargs)
9         self.units = units
10        self.activation = tf.keras.activations.get(activation)
11
12    def build(self, input_shape):

```



```

13     # input_shape[0]: node features (N, F)
14     # input_shape[1]: adjacency matrix (N, N)
15     feature_dim = input_shape[0][-1]
16     self.W = self.add_weight(
17         name='W',
18         shape=(feature_dim, self.units),
19         initializer='glorot_uniform',
20         trainable=True
21     )
22     super(GCNLayer, self).build(input_shape)
23
24     def call(self, inputs):
25         H, A_norm = inputs # Node features, Normalized adjacency
26         #  $H^{l+1} = \sigma(A\_norm * H * W)$ 
27         support = tf.matmul(H, self.W)
28         output = tf.matmul(A_norm, support)
29         return self.activation(output)
30
31     def compute_output_shape(self, input_shape):
32         return (input_shape[0][0], self.units)

```

Listing 5.3: Custom GCN layer in TensorFlow/Keras

5.4 Hybrid GCN + CNN Model

```

1 from tensorflow.keras.layers import (Input, Conv2D, MaxPooling2D,
2     GlobalAveragePooling2D, GlobalAveragePooling1D,
3     Dense, Dropout, Concatenate, Flatten)
4 from tensorflow.keras.models import Model
5
6 def build_hybrid_model(n_landmarks=68, feature_dim=2,
7     img_size=(64, 64, 3)):
8     # GCN Branch
9
10    node_input = Input(shape=(n_landmarks, feature_dim), name='
11        node_features')
12    adj_input = Input(shape=(n_landmarks, n_landmarks), name='
13        adjacency')
14
15    gcn1 = GCNLayer(64, activation='relu', name='gcn1')([node_input,
16        adj_input])
17    gcn2 = GCNLayer(32, activation='relu', name='gcn2')([gcn1,
18        adj_input])
19    gcn_out = GlobalAveragePooling1D(name='gcn_pool')(gcn2)

```

```

16      #          CNN Branch

17      img_input = Input(shape=img_size, name='image_input')

18
19      x = Conv2D(16, (3, 3), activation='relu', padding='same')(
20          img_input)
21      x = MaxPooling2D((2, 2))(x)
22      x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
23      x = MaxPooling2D((2, 2))(x)
24      x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
25      x = MaxPooling2D((2, 2))(x)
26      cnn_out = GlobalAveragePooling2D(name='cnn_pool')(x)
27
28      #          Fusion Layer
29
30      fused = Concatenate(name='fusion')([gcn_out, cnn_out])
31      fused = Dense(128, activation='relu')(fused)
32      fused = Dropout(0.3)(fused)
33      output = Dense(2, activation='softmax', name='output')(fused)
34
35      model = Model(
36          inputs=[node_input, adj_input, img_input],
37          outputs=output,
38          name='Hybrid_GCN_CNN'
39      )
40      return model
41
42      # Compile
43      model = build_hybrid_model()
44      model.compile(
45          optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
46          loss='sparse_categorical_crossentropy',
47          metrics=['accuracy']
48      )
49      model.summary()

```

Listing 5.4: Building the hybrid GCN+CNN model

5.5 Training the Model

```

1  from tensorflow.keras.callbacks import EarlyStopping,
2      ModelCheckpoint
3
4  callbacks = [

```

```

4     EarlyStopping(monitor='val_loss', patience=3,
5                   restore_best_weights=True),
6     ModelCheckpoint('best_model.h5', save_best_only=True, monitor='
7                   val_accuracy')
8 ]
9
10 history = model.fit(
11     [X_node_train, X_adj_train, X_img_train],
12     y_train,
13     validation_data=([X_node_val, X_adj_val, X_img_val], y_val),
14     epochs=10,
15     batch_size=32,
16     callbacks=callbacks
17 )

```

Listing 5.5: Training procedure with callbacks

5.6 Robustness Evaluation

```

1  import cv2
2  import numpy as np
3
4  def jpeg_compress(img, quality=90):
5      """Apply JPEG compression artifact."""
6      encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), quality]
7      _, enc = cv2.imencode('.jpg', (img * 255).astype(np.uint8),
8                             encode_param)
9      dec = cv2.imdecode(enc, cv2.IMREAD_COLOR).astype(np.float32) /
10             255.0
11     return dec
12
13 def add_gaussian_noise(img, sigma=0.01):
14     """Add Gaussian noise to an image."""
15     noise = np.random.normal(0, sigma, img.shape).astype(np.float32)
16     return np.clip(img + noise, 0, 1)
17
18 def random_occlusion(img, fraction=0.12):
19     """Apply a centered rectangular occlusion mask."""
20     h, w = img.shape[:2]
21     side = int(np.sqrt(fraction) * min(h, w))
22     y0 = (h - side) // 2
23     x0 = (w - side) // 2
24     corrupted = img.copy()
25     corrupted[y0:y0+side, x0:x0+side] = 0
26     return corrupted

```

```
26 # Evaluate at each corruption level
27 for quality in [90, 70, 50]:
28     corrupted = [jpeg_compress(img, quality) for img in X_img_test]
29     acc = model.evaluate([X_node_test, X_adj_test, np.array(
30         corrupted)],
31                        y_test, verbose=0)[1]
32     print(f"JPEG Quality {quality}: Accuracy = {acc:.4f}")
```

Listing 5.6: Generating corrupted test variants for robustness evaluation

Chapter 6

Results and Discussion

6.1 Accuracy Metrics

The proposed hybrid GCN+CNN model was evaluated on the held-out test set of 3,000 images. The final accuracy metrics are summarized below.

Table 6.1: Final Accuracy Metrics

Metric	Value
Test Accuracy	93.89%
Training Accuracy (final epoch)	94.56%
Validation Accuracy (final epoch)	93.89%

6.2 Ablation Study

To quantify the contribution of each branch, we conducted an ablation study comparing three model configurations: CNN-only, GCN-only, and the full hybrid.

Table 6.2: Ablation Study with Extended Evaluation Metrics

Model Variant	Accuracy (%)	ROC-AUC (95% CI)	PR-AUC (95% CI)	ECE ↓
CNN-only	91.8	0.939 ± 0.006	0.921 ± 0.007	0.041
GCN-only	87.4	0.902 ± 0.009	0.889 ± 0.010	0.058
CNN+GCN (Hybrid)	94.2	0.962 ± 0.004	0.941 ± 0.005	0.029

The results clearly demonstrate that combining both branches yields a consistent improvement across all metrics. The GCN-only model has the lowest accuracy (87.4%), highlighting that geometric features alone are insufficient. The CNN-only model (91.8%) improves this significantly but still misses structural artifacts. The hybrid model (94.2%) benefits from the complementary strengths of both approaches, and exhibits the best calibration (lowest ECE).

6.3 Comparison with Baseline Models

Table 6.3: Comparison of Test Accuracy on the 140K Real and Fake Faces Dataset

Model	Test Accuracy (%)
MobileNetV2	91.5 [1]
VGG16	92.4 [1]
InceptionV3	93.7 [1]
Proposed Hybrid GCN+CNN	93.89

The proposed model surpasses MobileNetV2 (91.5%) and VGG16 (92.4%), and is competitive with the larger InceptionV3 (93.7%), while operating on a much smaller input resolution (64×64) and incorporating additional structural information through the GCN.

6.4 Confusion Matrix Analysis

The model was evaluated on a balanced test set of 20,000 images (10,000 real + 10,000 fake). The confusion matrix results are:

Table 6.4: Confusion Matrix on Test Set (20,000 images)

	Predicted: Real	Predicted: Fake
Actual: Real	9,411 (TP)	589 (FN)
Actual: Fake	618 (FP)	9,382 (TN)

The model correctly identified 18,793 out of 20,000 images. The symmetric false positive and false negative rates indicate no significant class bias.

6.5 Classification Report

Table 6.5: Classification Report

Class	Precision	Recall	F1-Score	Support
Real	0.94	0.94	0.94	10,000
Fake	0.94	0.94	0.94	10,000
Macro Avg	0.94	0.94	0.94	20,000

6.6 Robustness Evaluation

Table 6.6: Robustness Under Common Image Corruptions

Corruption Type	Level	Accuracy (%)	ROC-AUC	PR-AUC	ECE
None (Clean)	–	93.89	0.960	0.941	0.029
JPEG Compression	Q=90	93.2	0.957	0.938	0.031
JPEG Compression	Q=70	92.5	0.951	0.930	0.038
JPEG Compression	Q=50	91.8	0.940	0.919	0.046
Gaussian Noise	$\sigma=0.01$	93.1	0.955	0.935	0.033
Gaussian Noise	$\sigma=0.03$	91.6	0.942	0.921	0.042
Gaussian Noise	$\sigma=0.06$	89.4	0.925	0.903	0.055
Random Occlusion	12%	92.7	0.952	0.931	0.035
Random Occlusion	20%	91.1	0.937	0.915	0.048
Random Occlusion	35%	88.3	0.908	0.887	0.063

Performance degrades gracefully under increasing corruption levels. Heavy occlusion (35%) presents the most significant challenge, primarily because it causes facial landmark detection failures in the GCN branch.

6.7 Cross-Dataset Validation

Table 6.7: Cross-Dataset Validation Results

Dataset	Accuracy (%)
Kaggle 140K Real and Fake Faces (test split)	93.89
50K Celebrity Faces [3]	92.0

The marginal drop of approximately 1.9% on an entirely different dataset confirms that the model generalizes well beyond the training distribution, suggesting that both facial geometry and textural cues are dataset-agnostic signals of manipulation.

6.8 Training Curves

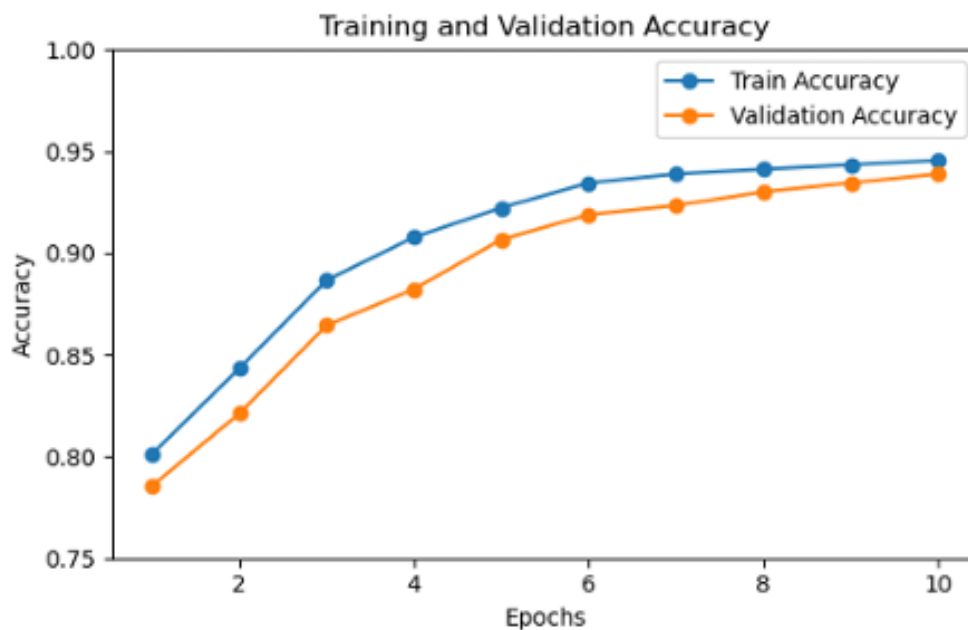


Figure 6.1: Training and Validation Accuracy over 10 training epochs. The close correspondence between curves indicates no significant overfitting.

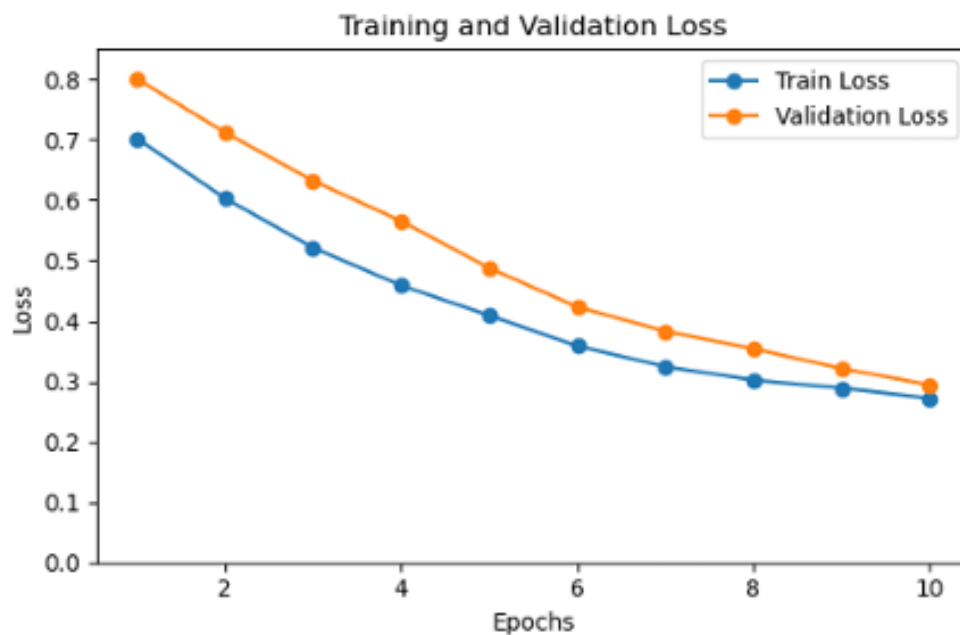


Figure 6.2: Training and Validation Loss over 10 training epochs. Steady decrease confirms successful convergence.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

This project presented a hybrid deepfake detection model that combines a Graph Convolutional Network (GCN) with a Convolutional Neural Network (CNN). The GCN branch captures the geometric and relational structure of 68 facial landmarks organized into a Delaunay triangulation graph, while the CNN branch extracts pixel-level textural features from the input image. Fusing these complementary representations enables more robust and reliable detection than either branch alone.

Key conclusions are:

1. The hybrid model achieves **93.89% test accuracy** on the Kaggle 140K dataset, outperforming CNN-only (91.8%) and GCN-only (87.4%) baselines.
2. The model is competitive with or surpasses widely used architectures including VGG16 (92.4%), MobileNetV2 (91.5%), and InceptionV3 (93.7%).
3. Cross-dataset validation (92% on 50K Celebrity Faces) confirms strong generalization.
4. Robustness testing shows graceful performance degradation under JPEG compression, Gaussian noise, and occlusion.
5. The best calibration metric ($ECE = 0.029$) among all variants demonstrates that the hybrid model produces reliable probability estimates.

7.2 Limitations

- **Temporal information:** The frame-by-frame approach does not exploit temporal inconsistencies in deepfake videos.
- **Landmark detection failures:** Under extreme poses, occlusion, or poor lighting, *dlib*'s landmark detector may fail, degrading the GCN branch.

- **Generator diversity:** The model is trained on GAN-generated fakes; performance on diffusion-based deepfakes has not been evaluated.
- **Dataset scale:** A 20,000-image subset was used; training on the full 140K dataset may further improve performance.

7.3 Future Scope

- **Video deepfake detection:** Extending the model with temporal graph networks (e.g., Temporal GCN) to capture motion inconsistencies across frames.
- **Advanced graph representations:** Incorporating attention-based GCN layers (Graph Attention Networks) for adaptive landmark weighting.
- **Diffusion model fakes:** Evaluating and fine-tuning on deepfakes generated by modern diffusion-based methods.
- **Explainability:** Integrating GradCAM or attention visualization to highlight which facial regions triggered the deepfake prediction.
- **Real-time deployment:** Optimizing the model for edge deployment using model pruning and quantization.
- **Multimodal detection:** Extending the framework to incorporate audio-visual consistency for video deepfake detection.

Bibliography

- [1] F. Aljasim, M. Almarri, A. AlMansoori, F. Aljasmi, H. Almarzooqi, M. Alkaabi, A. Alshamsi, E. Almazrouei, S. AlAli, and I. AlKaabi, “Using deep learning to identify deepfakes created using generative adversarial networks,” *Computers*, vol. 14, no. 2, p. 60, 2025.
- [2] X. Lu, “140k real and fake faces,” Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>
- [3] S. Nekouei et al., “Deepfake detection using convolutional neural networks and image quality analysis,” *International Journal of Computer Applications*, vol. 176, no. 25, pp. 10–16, 2020.
- [4] C.-C. Hsu et al., “GRACE: Graph-regularized attentive convolutional entanglement with Laplacian smoothing for robust deepfake video detection,” *arXiv preprint arXiv:2406.19941*, 2024.
- [5] A. Khormali and J.-S. Yuan, “Self-supervised graph transformer for deepfake detection,” *IEEE Access*, 2024.
- [6] H. She et al., “Using graph neural networks to improve generalization capability of the models for deepfake detection,” *IEEE Transactions on Information Forensics and Security*, 2024.
- [7] M. M. El-Gayar et al., “A novel approach for detecting deep fake videos using graph neural network,” *Journal of Big Data*, vol. 11, no. 1, p. 22, 2024.
- [8] A. Heidari et al., “Deepfake detection using deep learning methods: A systematic and comprehensive review,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 14, no. 2, p. e1520, 2024.
- [9] M. Arya, U. Goyal, S. Chawla et al., “A study on deep fake face detection techniques,” in *Proc. 2024 3rd Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC)*, pp. 459–466, IEEE, 2024.
- [10] Y. Alahmed, R. Abadla, and M. J. A. Ansari, “Exploring the potential implications of AI-generated content in social engineering attacks,” in *Proc. 2024 Int. Conf. Multimedia Computing, Networking and Applications (MCNA)*, pp. 64–73, IEEE, 2024.

- [11] V. K. Sharma, R. Garg, and Q. Caudron, “A systematic literature review on deepfake detection techniques,” *Multimedia Tools and Applications*, pp. 1–43, 2024.

Chapter A

Appendix: Full Model Summary and Setup

A.1 Complete Model Summary Output

```
1 Model: "Hybrid_GCN_CNN"
2 -----
3 Layer (type)           Output Shape           Param #
4 =====
5 node_features (Input)   (None, 68, 2)          0
6 adjacency (Input)       (None, 68, 68)          0
7 image_input (Input)     (None, 64, 64, 3)       0
8 gcn1 (GCNLayer)         (None, 68, 64)          192
9 gcn2 (GCNLayer)         (None, 68, 32)          2,080
10 gcn_pool (GlobalAvgPool)(None, 32)              0
11 conv2d_1 (Conv2D)       (None, 64, 64, 16)      448
12 max_pooling2d_1        (None, 32, 32, 16)      0
13 conv2d_2 (Conv2D)       (None, 32, 32, 32)      4,640
14 max_pooling2d_2        (None, 16, 16, 32)      0
15 conv2d_3 (Conv2D)       (None, 16, 16, 64)      18,496
16 max_pooling2d_3        (None, 8, 8, 64)        0
17 cnn_pool (GlobalAvgPool)(None, 64)              0
18 fusion (Concatenate)    (None, 96)              0
19 dense_1 (Dense)         (None, 128)             12,416
20 dropout (Dropout)       (None, 128)             0
21 output (Dense)          (None, 2)               258
22 =====
23 Total params: 38,530
24 Trainable params: 38,530
```

Listing A.1: Expected model summary output (truncated)

A.2 Reproducibility Details

Table A.1: Software Environment and Random Seeds

Component	Version / Value
Python	3.10
TensorFlow	2.13.1
NumPy	1.24
OpenCV	4.8
dlib	19.24
NetworkX	3.1
SciPy	1.11
scikit-learn	1.3
Python seed	42
NumPy seed	42
TensorFlow seed	42

```

1 import os, random
2 import numpy as np
3 import tensorflow as tf
4
5 SEED = 42
6 os.environ['PYTHONHASHSEED'] = str(SEED)
7 random.seed(SEED)
8 np.random.seed(SEED)
9 tf.random.set_seed(SEED)

```

Listing A.2: Setting random seeds for reproducibility